

Yes — your spec aligns almost perfectly with my plan. Below is the full deliverable covering sections A–E. I've read every relevant file already; no further inputs needed.

1) Summary of Current Issues / Risks

#	Issue	File(s)	Severity
1	Tokens in localStorage — Supabase JS default persistSession: true stores access + refresh in localStorage, vulnerable to XSS	supabaseClient.js	High
2	No 401 retry — axios has no response interceptor; expired token = silent failure	axios.js	High
3	Remote token validation on every request — authJWT calls supabase.auth.getUser(token) (HTTP to Supabase) per request; slow + fragile (ECONNRESET)	authMiddleware.js	Medium
4	No session bootstrap — page reload relies entirely on Supabase localStorage; no App.jsx auth recovery	App.jsx	High
5	No logout endpoint — sign-out is client-only (supabase.auth.signOut()); no server-side cookie clearing	Login.jsx	Medium
6	No CSRF mitigation — not an issue today (no cookies), but will be once refresh cookie exists	server.js	Medium
7	Missing deps — cookie-parser not installed; jose not installed (for local JWT verify)	package.json	Blocker
8	Missing env var — backend .env has no SUPABASE_JWT_SECRET (needed for local JWT verify)	.env	Blocker

2) Proposed Final Flow Diagram

===== LOGIN =====

Browser (Login.jsx)

Express (:3000)

Supabase

supabase.auth.signInWithPassword —————▶ /auth/v1/token

◀————— { access_token,
refresh_token }

accessToken → in-memory variable (axios module)

POST /api/auth/set-refresh —————▶ reads body.refresh_token

credentials: "include" res.cookie("sb_refresh", ..., { httpOnly, sameSite:"lax",
path:"/api/auth" })

◀————— { ok: true }

GET /api/webapp/users/profiles/me ▶ authJWT: jose.jwtVerify(Bearer token, JWT_SECRET)

Bearer <access_token> req.user = payload

◀————— { status: "active", ... }

navigate("/dashboard")

===== EVERY API CALL =====

axios request interceptor: attach in-memory access token as Bearer header

→ if 200: done

→ if 401:

POST /api/auth/refresh —————▶ reads cookie sb_refresh

withCredentials: true fetch Supabase /auth/v1/token?grant_type=refresh_token

◀—— { access_token, refresh_token (rotated) }
res.cookie("sb_refresh", new_refresh, ...)
◀———— { access_token, expires_in }
update in-memory token
retry original request

===== APP RELOAD (BOOTSTRAP) =====

App.jsx mount → POST /api/auth/refresh (withCredentials: true)
→ if ok: set in-memory access token → render app
→ if 401: no valid session → redirect /login

===== LOGOUT =====

POST /api/auth/logout —————▶ res.clearCookie("sb_refresh", { path: "/api/auth" })
credentials: "include"
◀———— { ok: true }
clear in-memory token
navigate("/login")

3) Step-by-Step Implementation Order

Step	Scope	What	Testable after?
1	Backend	npm i cookie-parser jose + add env vars	—
2	Backend	Wire cookieParser(), update CORS allowedHeaders in server.js	—
3	Backend	Add 3 endpoints in authRoutes.js / authController.js : set-refresh , refresh, logout + CSRF origin check	Postman
4	Backend	Switch authMiddleware.js to local JWT verify with jose	Postman
5	Frontend	Update supabaseClient.js : persistSession: false, autoRefreshToken: false	—
6	Frontend	Rewrite axios.js : in-memory token, withCredentials, 401 refresh+retry queue	—
7	Frontend	Update Login.jsx : after sign-in, POST refresh token to backend, store access token	Browser login
8	Frontend	Update App.jsx : bootstrapAuth() on mount	Browser reload

4) Exact Code Edits Per File

Step 1 — Install deps

```
cd backend
```

```
npm i cookie-parser jose
```

Step 2 — [.env](#) — add two vars

```
# existing vars stay...
```

```
SUPABASE_ANON_KEY=sb_publishable_KAIOg4NQffNgH-BxXsFLtA_FCOuBylF
```

SUPABASE_JWT_SECRET=<copy from Supabase Dashboard → Settings → API → JWT Secret>

SUPABASE_ANON_KEY value is the same as your existing [SUPABASE_API_KEY](#). We reference it by the standard name for the refresh endpoint. SUPABASE_JWT_SECRET is a **different value** — it's the HMAC signing secret from the dashboard (not the service role key).

Step 3 — [server.js](#) — wire cookie-parser + CORS update

Current top of [server.js:1-36](#):

// ADD after existing requires:

```
const cookieParser = require("cookie-parser");
```

// ADD before app.use(cors(...)):

```
app.use(cookieParser());
```

// UPDATE cors block:

```
app.use(
  cors({
    origin: (origin, callback) => {
      if (!origin || allowedOrigins.includes(origin)) {
        callback(null, true);
      } else {
        callback(new Error("CORS not allowed"));
      }
    },
    credentials: true,
    allowedHeaders: ["Content-Type", "Authorization"],
    methods: ["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
```

```
})  
);
```

Step 4 — [authController.js](#) — full rewrite

```
// backend/controllers/authController.js
```

```
const { createClient } = require("@supabase/supabase-js");
```

```
const supabaseAdmin = createClient(  
  process.env.SUPABASE_URL,  
  process.env.SUPABASE_SERVICE_ROLE_KEY  
);
```

```
// — cookie options —————
```

```
function refreshCookieOpts() {  
  const isProd = process.env.NODE_ENV === "production";  
  return {  
    httpOnly: true,  
    secure: isProd,      // true on HTTPS in prod; false on localhost  
    sameSite: "lax",     // blocks cross-site POST (CSRF baseline)  
    path: "/api/auth",   // cookie only sent to /api/auth/*  
    maxAge: 30 * 24 * 60 * 60 * 1000, // 30 days  
  };  
}
```

```
// — CSRF origin check —————
```

```
const allowedOrigins = ["http://localhost:5173"];
```

```

function assertOrigin(req) {
  const origin = req.headers.origin;
  if (!origin || !allowedOrigins.includes(origin)) {
    const err = new Error("Forbidden origin");
    err.status = 403;
    throw err;
  }
}

// — Supabase token refresh helper —————
async function supabaseRefresh(refreshToken) {
  const url = `${process.env.SUPABASE_URL}/auth/v1/token?grant_type=refresh_token`;
  const r = await fetch(url, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      apikey: process.env.SUPABASE_ANON_KEY,
    },
    body: JSON.stringify({ refresh_token: refreshToken }),
  });
  const json = await r.json();
  if (!r.ok) {
    const msg = json?.error_description || json?.error || "Refresh failed";
    throw new Error(msg);
  }
  return json; // { access_token, refresh_token, expires_in, ... }
}

```

```
}
```

```
// — POST /api/auth/login —————
```

```
// Kept for backward compat; frontend now signs in via Supabase JS directly,
```

```
// but this can be used by non-browser clients.
```

```
exports.login = async (req, res) => {
```

```
  try {
```

```
    const { email, password } = req.body;
```

```
    const {
```

```
      data: { session },
```

```
      error,
```

```
    } = await supabaseAdmin.auth.signInWithPassword({ email, password });
```

```
    if (error) return res.status(401).json({ error: error.message });
```

```
    res.json({ token: session.access_token, user: session.user });
```

```
  } catch (e) {
```

```
    res.status(500).json({ error: e.message });
```

```
  }
```

```
};
```

```
// — POST /api/auth/set-refresh —————
```

```
// Called once after frontend login to persist refresh token as HttpOnly cookie
```

```
exports.setRefresh = async (req, res) => {
```

```
  try {
```

```
    assertOrigin(req);
```

```
    const { refresh_token } = req.body;
```

```
    if (!refresh_token) {
```



```
    return res.status(400).json({ error: "Missing refresh_token" });
  }
  res.cookie("sb_refresh", refresh_token, refreshCookieOpts());
  return res.json({ ok: true });
} catch (e) {
  return res.status(e.status || 500).json({ error: e.message });
}
};
```

// — POST /api/auth/refresh —————

// Reads HttpOnly cookie, exchanges with Supabase for new tokens

```
exports.refresh = async (req, res) => {
```

```
  try {
```

```
    assertOrigin(req);
```

```
    const rt = req.cookies?.sb_refresh;
```

```
    if (!rt) return res.status(401).json({ error: "No refresh cookie" });
```

```
    const data = await supabaseRefresh(rt);
```

```
    // Rotate cookie if Supabase returned a new refresh token
```

```
    if (data.refresh_token) {
```

```
      res.cookie("sb_refresh", data.refresh_token, refreshCookieOpts());
```

```
    }
```

```
    return res.json({
```

```
      access_token: data.access_token,
```

```
    expires_in: data.expires_in,
  });
} catch (e) {
  // Clear bad cookie so user can re-login cleanly
  res.clearCookie("sb_refresh", { path: "/api/auth" });
  return res.status(401).json({ error: e.message || "Refresh failed" });
}
};

// — POST /api/auth/logout —————
exports.logout = async (req, res) => {
  res.clearCookie("sb_refresh", { path: "/api/auth" });
  return res.json({ ok: true });
};
```

Step 5 — [authRoutes.js](#) — add new routes

```
// backend/routes/authRoutes.js

const express = require("express");
const router = express.Router();
const authController = require("../controllers/authController");

router.post("/login", authController.login);
router.post("/set-refresh", authController.setRefresh);
router.post("/refresh", authController.refresh);
router.post("/logout", authController.logout);
```

```
module.exports = router;
```

Step 6 — [authMiddleware.js](#) — local JWT verify

```
// middleware/authMiddleware.js
```

```
const { jwtVerify } = require("jose");
```

```
const JWT_SECRET = new TextEncoder().encode(process.env.SUPABASE_JWT_SECRET);
```

```
exports.authJWT = async (req, res, next) => {
```

```
  const token = req.headers.authorization?.replace("Bearer ", "");
```

```
  if (!token) return res.status(401).json({ error: "No token" });
```

```
  try {
```

```
    const { payload } = await jwtVerify(token, JWT_SECRET, {
```

```
      issuer: `https://${process.env.SUPABASE_URL.replace("https://", "")}/auth/v1`,
```

```
    });
```

```
    // payload contains: sub (user id), email, role, exp, iat, etc.
```

```
    req.user = {
```

```
      id: payload.sub,
```

```
      email: payload.email,
```

```
      role: payload.role,
```

```
      aud: payload.aud,
```

```
    };
```

```
    next();
```

```
  } catch (err) {
```

```
    return res.status(401).json({ error: "Invalid or expired token" });
```

```
}  
};
```

Step 7 — [supabaseClient.js](#) — disable persistence

```
import { createClient } from "@supabase/supabase-js";  
  
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;  
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;  
  
export const supabase = createClient(supabaseUrl, supabaseAnonKey, {  
  auth: {  
    persistSession: false,  
    autoRefreshToken: false,  
    detectSessionInUrl: true, // keep for password reset / magic links  
  },  
});
```

Step 8 — [axios.js](#) — in-memory token + refresh-on-401

```
// src/api/axios.js  
  
import axios from "axios";  
  
const api = axios.create({  
  baseURL: "http://localhost:3000/api",  
  headers: { "Content-Type": "application/json" },  
  withCredentials: true, // send cookies on every request (needed for /auth/refresh)  
});
```

// — In-memory access token —————

```
let accessToken = null;
```

```
export function setAccessToken(token) {  
  accessToken = token;  
}
```

```
export function getAccessToken() {  
  return accessToken;  
}
```

// — Request interceptor: attach Bearer —————

```
api.interceptors.request.use((config) => {  
  if (accessToken) {  
    config.headers.Authorization = `Bearer ${accessToken}`;  
  }  
  return config;  
});
```

// — Response interceptor: 401 → single refresh → retry —

```
let isRefreshing = false;
```

```
let refreshQueue = []; // queued requests waiting for refresh
```

```
api.interceptors.response.use(  
  (res) => res,  
  async (err) => {
```

```
const original = err.config;

const status = err.response?.status;

// Only intercept 401, only retry once, skip refresh endpoint itself
if (status !== 401 || original._retry || original.url === "auth/refresh") {
  return Promise.reject(err);
}

original._retry = true;

if (isRefreshing) {
  // Another call is already refreshing — queue this one
  return new Promise((resolve, reject) => {
    refreshQueue.push({ resolve, reject });
  }).then((newToken) => {
    original.headers.Authorization = `Bearer ${newToken}`;
    return api(original);
  });
}

isRefreshing = true;

try {
  const r = await api.post("auth/refresh"); // cookie sent automatically
  const newToken = r.data.access_token;
  setAccessToken(newToken);
}
```

```

// Resolve all queued requests
refreshQueue.forEach(({ resolve }) => resolve(newToken));
refreshQueue = [];

original.headers.Authorization = `Bearer ${newToken}`;
return api(original);
} catch (refreshErr) {
  // Refresh failed — force logout
  refreshQueue.forEach(({ reject }) => reject(refreshErr));
  refreshQueue = [];
  setAccessToken(null);

  // Redirect to login (avoid import cycle — use window.location)
  window.location.href = "/login";
  return Promise.reject(refreshErr);
} finally {
  isRefreshing = false;
}
}
);

export default api;

```

Step 9 — [Login.jsx](#) — post-login: set cookie + memory token

```
// Login.jsx
```

```
import './Login.css';

import { useState } from 'react';

import { supabase } from '../lib/supabaseClient';

import api, { setAccessToken } from '../api/axios';

import { Link, useNavigate } from 'react-router-dom';

const Login = () => {

  const [email, setEmail] = useState('');

  const [password, setPassword] = useState('');

  const [error, setError] = useState(null);

  const [loading, setLoading] = useState(false);

  const navigate = useNavigate();

  const handleSubmit = async (e) => {

    e.preventDefault();

    setError(null);

    setLoading(true);

    try {

      // 1) Sign in via Supabase JS (gets tokens but NOT persisted)

      const { data, error: authError } = await supabase.auth.signInWithPassword({

        email,

        password,

      });

      if (authError) {
```



```
setError(authError.message);  
setLoading(false);  
return;  
}
```

```
const session = data.session;  
if (!session) throw new Error("No session returned");
```

```
// 2) Store access token in memory for axios  
setAccessToken(session.access_token);
```

```
// 3) Send refresh token to backend → HttpOnly cookie  
await api.post("auth/set-refresh", {  
  refresh_token: session.refresh_token,  
});
```

```
// 4) Check profile status (uses Bearer from memory now)  
const res = await api.get("webapp/users/profiles/me");  
const profile = res.data;
```

```
if (profile.status !== "active") {  
  await api.post("auth/logout");  
  setAccessToken(null);  
  setError(  
    "Your account is on hold or inactive. Please contact the administrator."  
  );  
}
```

```

    setLoading(false);
    return;
  }

  // 5) All good — navigate
  navigate("/dashboard");
} catch (err) {
  console.error("Login failed:", err);
  setAccessToken(null);
  setError(
    err?.response?.data?.error ||
    err.message ||
    "Login failed. Please try again."
  );
  setLoading(false);
}
};

return (
  <div className="login-page">
    <div className="login-card">
      <h1 className="login-title">Login to your account</h1>
      <form className="login-form" onSubmit={handleSubmit}>
        <div className="form-group">
          <label>Email</label>
          <input

```

```
    type="email"
    placeholder="Enter your email"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
    required
  />
</div>
<div className="form-group">
  <label>Password</label>
  <input
    type="password"
    placeholder="Enter your password"
    value={password}
    onChange={(e) => setPassword(e.target.value)}
    required
  />
</div>
<p className="forgot-password-text">
  <Link to="/forgot-password">Forgot your password?</Link>
</p>
{error && <p className="error-text">{error}</p>}
<button type="submit" className="login-button" disabled={loading}>
  {loading ? "Logging in..." : "Login now"}
</button>
</form>
</div>
```

```
    </div>

  );

};

export default Login;
```

Step 10 — [App.jsx](#) — bootstrap auth on mount

```
import { useState, useEffect } from "react";
import { BrowserRouter as Router, Routes, Route, Navigate } from "react-router-dom";
import Sidebar from "./layouts/Sidebar";
import Dashboard from "./pages/Dashboard/Dashboard";
import SAM from "./pages/SAM/SAM";
import DeviceManagement from "./pages/DeviceManagement/DeviceManagement";
import AccountsAudit from "./pages/AccountsAudit/AccountsAudit";
import History from "./pages/History/History";
import Profile from "./pages/Profile/Profile";
import Login from "./pages/Login/Login";
import ForgotPassword from "./pages/Auth/ForgotPassword";
import ResetPassword from "./pages/Auth/ResetPassword";
import api, { setAccessToken, getAccessToken } from "./api/axios";
import "./App.css";

function App() {
  const [authReady, setAuthReady] = useState(false);

  useEffect(() => {
```

```

// On mount: try to restore session from refresh cookie

api
  .post("auth/refresh")
  .then((res) => {
    setAccessToken(res.data.access_token);
  })
  .catch(() => {
    setAccessToken(null); // no valid session — will redirect to login
  })
  .finally(() => {
    setAuthReady(true);
  });
}, []);

if (!authReady) {
  return <div className="loading-screen">Loading...</div>;
}

const isAuthenticated = !!getAccessToken();

return (
  <Router>
    <Routes>
      <Route path="/" element={<Navigate to="/login" replace />} />
      <Route path="/login" element={<Login />} />
      <Route path="/forgot-password" element={<ForgotPassword />} />
    </Routes>
  </Router>
)

```

```
<Route path="/reset-password" element={<ResetPassword />} />
```

```
{/* Protected routes */}
```

```
<Route
```

```
  path="/*"
```

```
  element={
```

```
    isAuthenticated ? (
```

```
      <div className="app">
```

```
        <Sidebar />
```

```
        <main className="main-content">
```

```
          <Routes>
```

```
            <Route path="/dashboard" element={<Dashboard />} />
```

```
            <Route path="/security-assessment" element={<SAM />} />
```

```
            <Route path="/device-management" element={<DeviceManagement />} />
```

```
            <Route path="/accounts-audit" element={<AccountsAudit />} />
```

```
            <Route path="/history" element={<History />} />
```

```
            <Route path="/profile" element={<Profile />} />
```

```
          </Routes>
```

```
        </main>
```

```
      </div>
```

```
    ) : (
```

```
      <Navigate to="/login" replace />
```

```
    )
```

```
  }
```

```
</Routes>
```

```

    </Router>

  );
}

export default App;

```

5) Failure Modes & Handling (Deliverable D)

Scenario	What happens	User sees
Access token expired	axios 401 → response interceptor calls /api/auth/refresh → new token → retries original request	Nothing — seamless
Refresh token revoked / expired	/api/auth/refresh returns 401 → cookie cleared server-side → setAccessToken(null) → window.location.href = "/login"	Redirected to login
Supabase down	supabaseRefresh() fetch fails → backend returns 500/502 → same as revoked flow	"Login failed" or redirect to login
Multiple concurrent 401s	First caller sets isRefreshing = true; others queue. Single refresh call, all resolved with new token or all rejected	Seamless or single redirect
Cookie not sent (CORS/SameSite)	req.cookies.sb_refresh is undefined → 401 "No refresh cookie" → redirect to login	Redirected to login
XSS tries to read refresh token	httpOnly: true → JS cannot access cookie. Access token in memory only — survives page lifetime, not extractable from storage	Attacker gets nothing persistent

6) Test Plan Checklist (Deliverable E)

Postman (backend only, after steps 1–4)

- [POST /api/auth/login](#) with `{ email, password }` → returns `{ token, user }`
- [POST /api/auth/set-refresh](#) with `{ refresh_token: "<from login>" }` → returns `{ ok: true }` + `Set-Cookie: sb_refresh=...`
- [POST /api/auth/refresh](#) with cookie from above → returns `{ access_token, expires_in }` + rotated `Set-Cookie`
- [POST /api/auth/refresh](#) without cookie → 401 "No refresh cookie"
- [POST /api/auth/refresh](#) with expired/revoked refresh token → 401 + cookie cleared
- [POST /api/auth/logout](#) → `{ ok: true }` + cookie cleared
- [GET /api/webapp/users/profiles/me](#) with valid Bearer → 200 + profile
- [GET /api/webapp/users/profiles/me](#) with expired Bearer → 401

Browser (after all steps)

- **Login:** sign in → dashboard loads → DevTools Application tab: no `sb-*` keys in `localStorage`
- **Network tab:** [/set-refresh](#) response has `Set-Cookie: sb_refresh=...; HttpOnly; Path=/api/auth`
- **Reload dashboard:** brief loading screen → [/auth/refresh](#) called → dashboard renders (session restored)
- **Idle until access token expires** (~1 hr default): next API call triggers 401 → refresh → retry → works
- **Multiple tabs:** open 2 tabs, let token expire, click in both — only 1 refresh call (queue works)
- **Logout:** click logout → cookie cleared → reload → redirected to login
- **Revoke refresh token** (from Supabase dashboard): reload → [/auth/refresh](#) fails → redirected to login

Pitfalls checklist

- `withCredentials: true` on axios instance (not just individual calls)
- CORS `credentials: true` + explicit origin (not `*`)

- CORS allowedHeaders includes Content-Type and [Authorization](#)
- Cookie [path: "/api/auth"](#) matches the refresh/logout endpoint paths
- Cookie secure: false for localhost dev; secure: true in prod
- Cookie sameSite: "lax" (not "none" unless cross-site needed)
- Do NOT set domain for localhost
- clearCookie uses the same [path](#) as [res.cookie](#)
- _retry flag on axios to prevent infinite 401 loops
- Refresh endpoint excluded from 401 interceptor (original.url === "auth/refresh")

This is the complete plan. It covers all 5 deliverables (A–E) and is implementable file-by-file in the order listed. Want me to start coding it into your repo? I'd begin with the backend (steps 1–4) so you can verify with Postman before we touch the frontend.